Exercises for Weeks 1–2

Programming DT2012

Fall 2025

1 Introduction

This assignment focuses on **data types and operators** and **control flow and loops**. Each part has two activities that you need to complete. These exercises cover the topics from lectures 1 and 2. All instructions for each activity are in sections 2 and 3.

After completing the assignment, you will submit your files through Blackboard. The submission package **must** be a compressed directory (i.e., a zip file) with **only** the Java files for each activity within it. The compressed directory **must** have the same name as your student acronym (i.e., magols25), and the Java files **must** be called A1.java - A4.java, where A1.java is the first exercise, A2.java the second, A3.java the third, and A4.java is the last.

Read the complete document before submitting your work. By submitting your work through Blackboard or any other means, it is assumed by the staff that you have read and understood the complete assignment, its constraints, its rules and its implications.

2 Data types and operators

Objectives

By the end of this part of the lab

- You will be able to translate a problem from human language into variables and commands in a program.
- You will be able to identify the adequate basic data type to represent a variable from reality.
- You will be able to use operators within the same data type.
- You will be able to use operators between different data types and know which type of result to expect.

Activity 1: days to goal

According to statistics, it takes between 400 to 2,200 hours of study plus real-world practice to learn a language (e.g., Swedish, Spanish). Popular knowledge says that it takes 10,000 hours to become an expert at something, like playing a musical instrument. Most aviation authorities require a minimum of 1,500 hours of flight to provide a license as a pilot. However, we might struggle to find out the actual calendar time it will take us to acquire an ability depending on the daily effort we can put into it.

Write a program that takes the expected time in hours to complete a skill and the time in hours you can put into it and write how many years/days/hours/minutes it will take you to master that skill.

Requirements and Constraints

- The program will take the amount of whole hours to skill completion as input
- The amount of time one can dedicate each day can be expressed in whole hours (e.g., 4) and include half-hours (e.g., 4.5, 0.5) or even quarter hours (e.g., 0.25). Make sure you understand what 0.25 hours equals in minutes.
- A *year* is 365 *days*. A *month* is 30 *days*. I.e., we do not deal with leap years and/or months of different duration.
- You cannot use any pre-defined functions. You can only use operators for the basic data types (e.g., +,-,%), operations from the String class (*java.lang.String*), operations to convert a variable's datatype and print functions (*System.out.println* & *System.out.print*).
- The program must print out:

  Hours to completion: ⟨number of hours⟩

  Daily dedication: ⟨*hours*⟩:⟨*minutes*⟩

  It will take you ⟨y⟩ years, ⟨m⟩ months, ⟨d⟩ days, ⟨h⟩ hours, ⟨m⟩ minutes to reach the goal at this rate

- If any of the values is zero, that gets omitted from the output (i.e., if it only takes less than a month but whole days but no extra hours or minutes, it will only print out *It will take you* ⟨d⟩ *days to reach the goal at this rate*)

Example run

```
$ java A1 10000 8
  Hours to completion: 10000
  Daily dedication: 8:00
  It will take you 3 years, 5 months, 5 days to reach
```

```
  the goal at this rate
$ java A1 10000 8.5
  Hours to completion: 10000
  Daily dedication: 8:30
  It will take you 6 years, 1 month, 1 day, 2 hours to
  reach the goal at this rate
```

Activity 2: create a HH Blackboard name

You have an username that you can use to access Halmstad University's (HH) digital services such as Blackboard or e-mail. For most cases, you probably have noticed that it is a lowercase word that consists of the first three letters of your given name, the first three letters of your family name, followed by the 2-digit format of the year you registered at HH for the first time. For the case of your instructor, Oscar Molina, who arrived in 2021, the username would be *oscmol21*.

Write a program that takes a first name, a family name, and a year, and delivers a user name and a student e-mail address.

Requirements and Constraints

- The program will take input in the Swedish alphabet but deliver outputs in the 26-letter latin alphabet.
- Characters å, ä, ö are converted to a, a and o respectively.
- The first- and last name must have at least 3 characters.
- The year must have exactly 4 integers (e.g. 2025, 1973).
- The input can be provided in UPPERCASE, lowercase, or AnY CoMbInAtIoN of upper- and lower-case letters.
- You cannot use any pre-defined functions. You can only use operators for the basic data types (e.g., +,-,%), operations from the String class (*java.lang.String*), operations to convert a variable's datatype and print functions (*System.out.println* & *System.out.print*).
- The program must print out:
  - Username: ⟨first three letters of name⟩⟨first three letters of surname⟩⟨last two digits of the year⟩
  - E-mail address: ⟨*username*⟩@student.hh.se

Example run

```
$ java A2 Oscar Molina 2021
  Username: oscmol21
  E-mail address: oscmol21@student.hh.se
$ java A2 ernesto fonseca 2023
  Username: ernfon23
  E-mail address: ernfon23@student.hh.se
```

3 Control Flow and Loops

Objectives

By the end of this part of the lab

- You will be able to translate a problem from human language into variables, commands, and control flow, and loops in a program.
- You will be able to identify the adequate loop and control flow statements to represent events from reality.
- You will be able to use pre-defined functions that create random numbers.

Activity 3: simulating bets with fixed-odds

According to Wikipedia:

> Fixed-odds betting is a form of gambling where individuals place bets on the outcome of an event, such as sports matches or horse races, at predetermined odds. In fixed-odds betting, the odds are fixed and determined at the time of placing the bet. These odds reflect the likelihood of a particular outcome occurring. If the bettor's prediction is correct, they receive a payout based on the fixed odds.

Let us imagine that you bet for a sports team for a whole season consisting of n matches. You wager an original bet b per game. The bet is differentiated by home and away matches (you tend to bet more for home matches). The matches occurring at odd matchdays (i.e., 1, 3, 5, 7, ...) are home matches and the team has a 66% chance of winning, 24% chance of getting a tie, and 10% chance of losing. The matches occurring at even matchdays (i.e., 2, 4, 6, ...) are away games, and the team has a 30% chance of winning, 30% chance of getting a tie, and 40% chance of losing.

Home match wins have odds of 1.5 (you get an additional SEK 0.50 for every SEK 1 that you bet), while a tie or a loss makes you lose the SEK 1. Wins as an away team have odds of 3.25 (you get an additional SEK 2.25 for every SEK 1 you bet). You add the profit or subtract the loss from your balance.

Write a program that takes a number of matches, a bet in SEK for home games and a bet in SEK for away games. The output is the count of home and away wins, ties and loses, and the final balance of money.

Requirements and Constraints

- The structure for the input is *java A3 n h a* where n is the number of matches, h is the wager for home matches, and a is the wager for away matches.
- The program will take wagers in krona and öre (e.g., 99.99) as input.
- It is recommended that the value for h is larger than the one for a, but this is not enforced, since there is a larger payout for away games.
- You cannot use any pre-defined functions. You can only use operators for the basic data types (e.g., +,-,%), operations to convert a variable's datatype, print functions (*System.out.println* & *System.out.print*), and *Math.random*().
- The program must print out:
  - Home: *hw* wins, *ht* ties, *hl* loses.
  - Away: *aw* wins, *at* ties, *al* loses.
  - Balance: SEK *b*

Example run

```
$ java A3 38 10 5
  Home: 12 wins, 4 ties, 2 loses
  Away: 5 wins, 5 ties, 8 loses.
  Balance: SEK -20.00
$ java A3 40 10 10
  Home: 14 wins, 4 ties, 2 loses
  Away: 6 wins, 6 ties, 8 loses.
  Balance: SEK 2.50
  $ java A3 30 100 20
  Home: 12 wins, 2 ties, 1 loses
  Away: 7 wins, 3 ties, 5 loses.
  Balance: SEK 430.00
```

Activity 4: converting base-10 numbers to other bases

One of the perks of working in the area you are starting in (i.e., engineering), is the joy of using other numbering systems — binary, hexadecimal, sexagesimal.

You will find yourselves trying to convert a decimal number (usually a positive integer) into another base (like base-2 for binary, or base-16 for hexadecimal).

Write a program that takes a positive integer n in the base-10 system and another base b (up to base-26) and prints out the original number and its conversion and base.

Requirements and Constraints

- Validate the input so that your program only reacts to positive integers (e.g., reject -20 or 2.141592654).
- All bases start with zero as their first number, so, the largest symbol will represent $b - 1$.
- Numbers can only represent values 0–9. In hexadecimal, letters A–F are used to represent 10–15. For bases beyond 10, use capital letters to complement the symbols.
- You cannot use any pre-defined functions. You can only use operators for the basic data types (e.g., +,-,%), operations to convert a variable's datatype, print functions (*System.out.println* & *System.out.print*), *Math.pow*() and *Math.random*().
- The program will print out
  - n in base-10 is c in base-b.

Example run

```
$ java A4 10 2
  10 in base-10 is 1010 in base-2
$ java A4 3 3
  3 in base-10 is 10 in base-3
$ java A4 48879 16
  48879 in base-10 is BEEF in base-16
```

4 Grading criteria

Each activity is evaluated as a unit. There are 4 points at play, and you must get at least 3 to pass this set of exercises (75%). To pass each unit, the program must compile and deliver the expected output depending on the arguments that are provided. The code must comply with **all** the requirements and constraints described in the activity.

To be awarded the points, you must hand in original code that was designed and typed by yourself. Code that does not pass the originality or similarity tests we subject it to will be submitted to the disciplinary board. Even if you work in teams or collaborate, each student must submit original code. Code that was obtained from Generative AI assistants is not original code.

Points are awarded after the mandatory oral examination. In this examination, you will answer specific questions about your code, explain how changing some commands affects performance, and general questions about Java and programming. You can bring your own pseudocode or activity diagrams to the oral examination as aids.

Honor Pledge

By submitting this assignment, **you affirm that you have not given or received any unauthorized help on this assignment, and that this work is your own**. Unauthorized aids include, but are not limited to:

- Using or submitting text, code, or other results that was obtained from Generative AI and Large Language Models (e.g., ChatGPT, Copilot).
- Using or submitting text, code, or other results obtained from peers in the same class, from students in higher classes, from students in other universities, or people outside the university.
- Showing or lending text or code to peers in your class (both parties become liable to be submitted to the disciplinary board).

All work is to be performed individually, as it is going to be evaluated individually. There are no group tasks in the assignments you will get, and *collaborating with peers or other students* does not justify the submission of work that is not original and made by the student submitting it.

Any violation to this pledge is considered cheating and will be reported to the Vice-Chancellor in accordance to Chapter 10, section 1 point 1 of the Higher Education Ordinance.

Tasks from old exams that can be solved after activities 1 and 2.

The blue text is a solution proposal.

- For the following three expressions, specify the value and type along with a brief explanation.

Expression	Value	Type	Explanation
`1 + ",/ ", + 2 + ", = ", + 1 / 2`	`",1 / 2 = 0"`	String	See below the table
`Integer.parseInt("123") * Double.parseDouble("2")`	`246.0`	double	See below the table
`(int)Math.random()`	`0`	int	See below the table

Explanation regarding the first row: the precedence and associativity of operations are such that the expression stands for `(((1 + ",/ ",) + 2 ) + ", = ",) + (1 / 2)`. The + operation between String and int converts the integer to String and concatenates both Strings. The / operation between integers is the quotient.
Explanation regarding the second row: arithmetic operations between int and double convert the integer to double.
Explanation regarding the third row: `Math.random()` is a double in the interval [0, 1). Type conversion (int) retains only the integer part (ignores all decimals).

- What will be the output of the following program:

```
public static void main(String[] cmdLn){
System.out.println(5+3);
System.out.println(5-3);
System.out.println(53);
System.out.println(5/3);
System.out.println(5%3);
System.out.println(35-2);
System.out.println(3+5/2);
System.out.println(3-5-2);
System.out.println((3-5)-2);
System.out.println(3-(5-2));
System.out.println(5/3*2);
}
```

- What will be the output of the following program:

```
public static void main(String[] cmdLn){
String ruler1 = "1";
String ruler2 = ruler1 + " 2 " + ruler1;
String ruler3 = ruler2 + " 3 " + ruler2;
String ruler4 = ruler3 + " 4 " + ruler3;
```

```
String ruler5 = ruler4 + " 5 " + ruler4;
System.out.println(ruler1);
System.out.println(ruler2);
System.out.println(ruler3);
System.out.println(ruler4);
System.out.println(ruler5);
}
```

Tasks from old exams that can be solved after activities 3 and 4.

The blue text is a solution proposal.

- Consider the following code fragment:

```
for(int i = 0; i < 5 ; i++) {
    for(int j = i; j < 5; j++) {
        System.out.print("(" + i + ", " + j + ") ");
        }
    System.out.println();
}
```

1. What are the control variables in the for-loops and what values do they take when the code fragment is executed?
   The control variables are i and j. When the code fragment is executed, i takes the values 0, 1, 2, 3, and 4 in that order. j takes the following values:
   when i takes the value 0, j takes the values 0, 1, 2, 3, and 4,
   when i takes the value 1, j takes the values 1, 2, 3, and 4,
   when i takes the value 2, j takes the values 2, 3, and 4,
   when i takes the value 3, j takes the values 3 and 4,
   when i takes the value 4, j takes the value 4.
2. Specify how many times the command `System.out.print("(" + i + ", " + j + ") ");` is executed.
   The command is executed 5 + 4 + 3 + 2 + 1 times (15 times)
3. Specify what the output looks like.
   (0, 0) (0, 1) (0, 2) (0, 3) (0, 4)
   (1, 1) (1, 2) (1, 3) (1, 4)
   (2, 2) (2, 3) (2, 4)
   (3, 3) (3, 4) (4, 4)